Name: Farman Ilahi A
Reg No: 192511363
Course: CSA0618 – Design and Analysis of Algorithm
Date: 05-10-2026

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω: O(1), O: O(n), Θ: Θ(n)
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

Answer:

Aim

To analyze the time complexity of linear search and search for a key in an array.

Algorithm

1. Start.
2. Initialize the array and search key.
3. Traverse the array from the first element.
4. Compare each element with the key.
5. If the key is found, display its index.
6. If the key is not found, display "Key not found".
7. Stop.

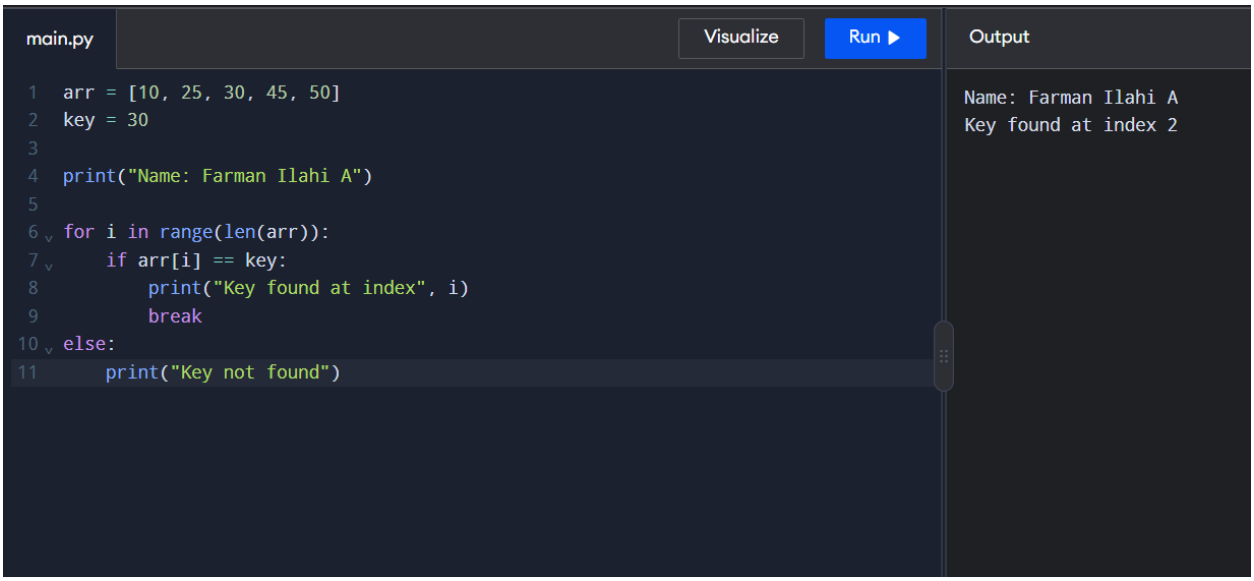
Python Program

```
arr = [10, 25, 30, 45, 50]

key = 30

print("Name: Farman Ilahi A")

for i in range(len(arr)):
    if arr[i] == key:
        print("Key found at index", i)
        break
else:
    print("Key not found")
```



Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω: O(1), O: O(log$_{f_0}$n), Θ: Θ(log$_{f_0}$n)
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

Answer:

Aim

To implement recursive binary search and analyze its complexity.

Algorithm

1. Start.
2. Set the lower and upper limits.
3. Find the middle element.
4. Compare the middle element with the key.
5. If equal, return the index.
6. If the key is smaller, search the left half.
7. Otherwise, search the right half.
8. Repeat recursively until the key is found or the range becomes empty.
9. Stop.

Python Program

```
def binary_search(arr, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)
arr = [5, 10, 15, 20, 25]
key = 20
result = binary_search(arr, 0, len(arr) - 1, key)
print("Name: Farman Ilahi A")
if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

main.py Visualize Run ▶ Output

```python
def binary_search(arr, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)


arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr) - 1, key)

print("Name: Farman Ilahi A")

if result != -1:
```

```
Name: Farman Ilahi A
Key found at index 3
```

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** n = 5
- **Sample Output:** 120
- **Complexity:** Both O(n)
- **Explanation:** Highlights recursion depth vs iterative loops.

Answer:

Aim

To compare iterative and recursive methods for computing factorial.

Algorithm

Iterative:

1. Start.
2. Read n.

3. Initialize factorial as 1.
4. Multiply factorial by each number from 1 to n.
5. Display the result.

Recursive:

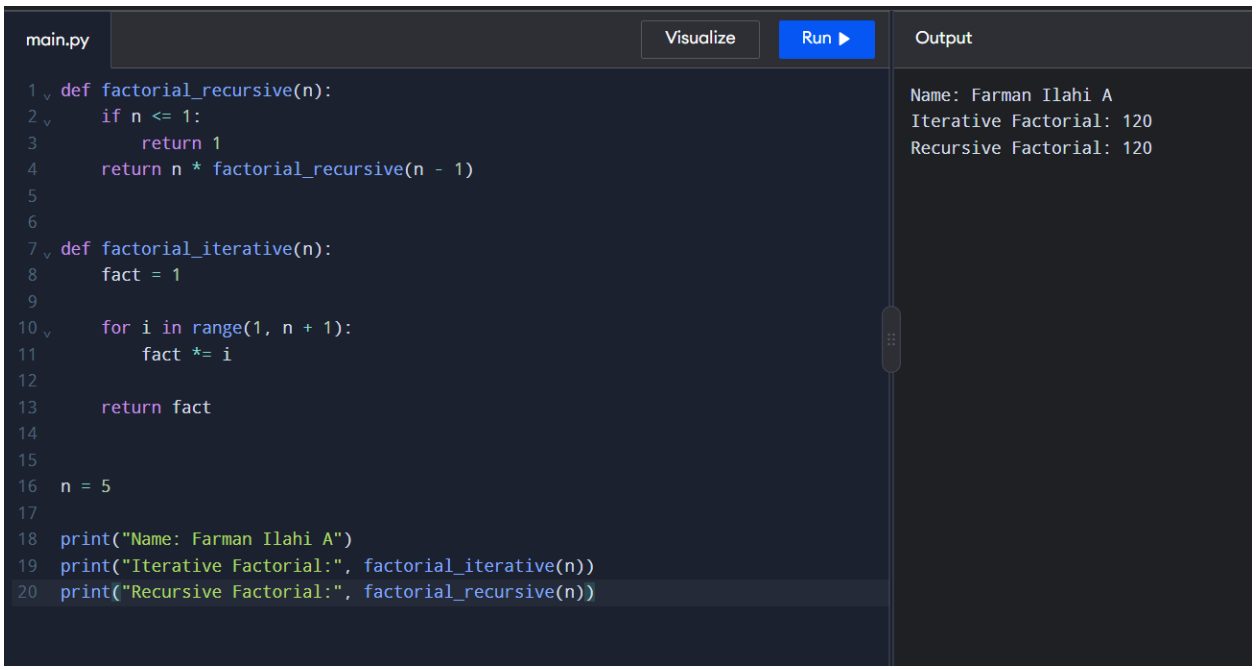
1. If n is 0 or 1, return 1.
2. Otherwise, return n × factorial(n-1).
3. Display the result.
4. Stop.

Python Program

```
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)
def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
 return fact
n = 5
print("Name: Farman Ilahi A")
```

print("Iterative Factorial:", factorial_iterative(n))

print("Recursive Factorial:", factorial_recursive(n))



Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

Answer:

Aim

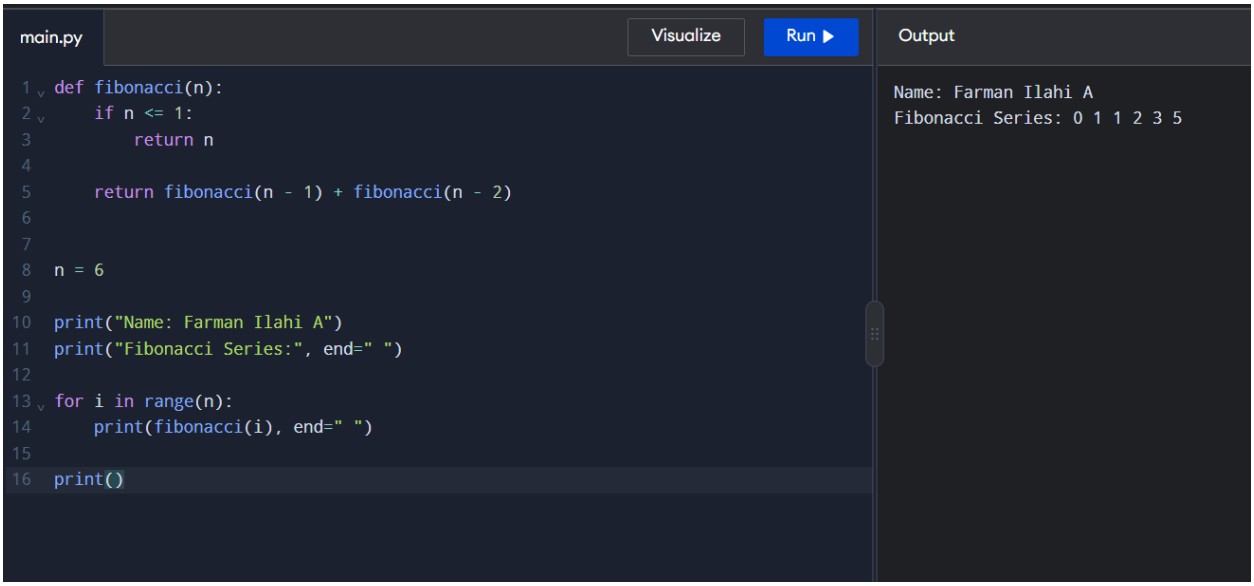
To analyze recursive and iterative Fibonacci series generation.

Algorithm

1. Start.
2. Read the number of Fibonacci terms.
3. Define the Fibonacci function recursively.
4. If n is 0 or 1, return n.
5. Otherwise, return the sum of the previous two Fibonacci numbers.
6. Generate the required number of terms.
7. Display the series.
8. Stop.

Python Program

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
n = 6
print("Name: Farman Ilahi A")
print("Fibonacci Series:", end=" ")
for i in range(n):
   print(fibonacci(i), end=" ")
print()
```



Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** A=[[1,2],[3,4]], B=[[5,6],[7,8]]
- **Sample Output:** [[19,22],[43,50]]
- **Complexity:** O(n3)
- **Explanation:** Shows cubic growth due to nested loops.

Answer:

Aim

To analyze the complexity of matrix multiplication using nested loops.

Algorithm

1. Start.
2. Initialize matrices A and B.
3. Create a result matrix with zeros.
4. Use three nested loops.

5. Multiply corresponding elements.
6. Add the products to the result matrix.
7. Display the result matrix.
8. Stop.

Python Program

```
A = [[1, 2],
    [3, 4]]
B = [[5, 6],
    [7, 8]]
C = [[0, 0],
    [0, 0]]
for i in range(2):
   for j in range(2):
       for k in range(2):
           C[i][j] += A[i][k] * B[k][j]
print("Name: Farman Ilahi A")
print("Result Matrix:")
for row in C:
   print(row)
```

main.py Visualize Run ▶ Output

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

C = [[0, 0],
     [0, 0]]

for i in range(2):
    for j in range(2):
        for k in range(2):
            C[i][j] += A[i][k] * B[k][j]

print("Name: Farman Ilahi A")
print("Result Matrix:")

for row in C:
    print(row)
```

```
Name: Farman Ilahi A
Result Matrix:
[19, 22]
[43, 50]
```

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always O(nlogf_0n)
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

Answer:

Aim

To implement merge sort and analyze divide-and-conquer sorting.

Algorithm

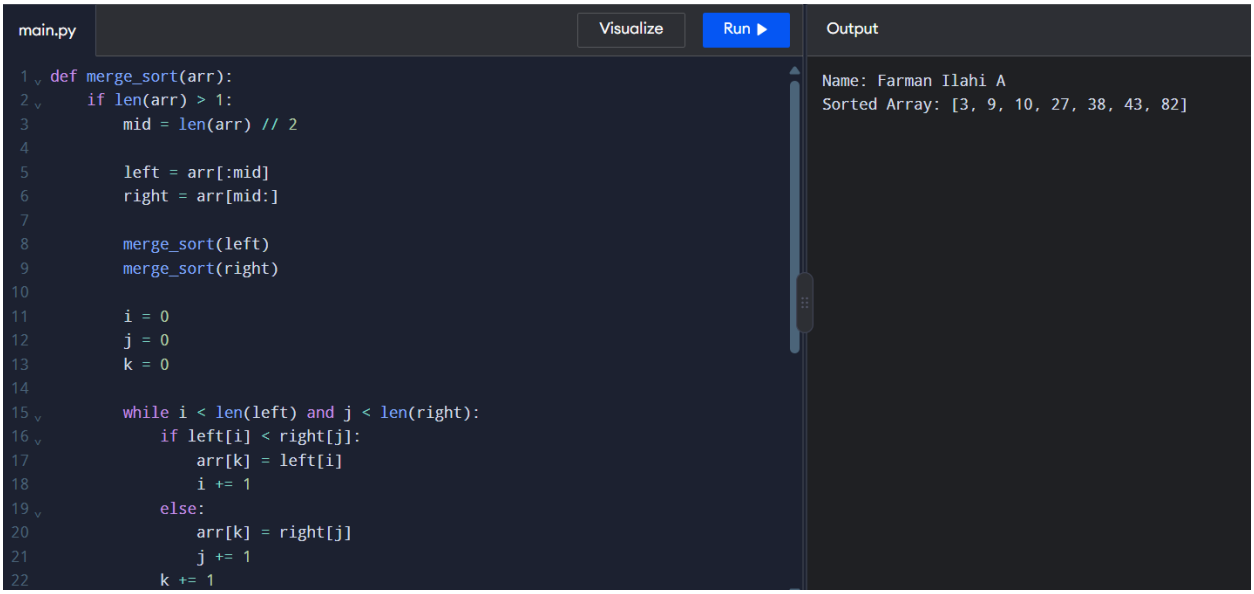
1. Start.
2. Divide the array into two halves.
3. Recursively divide each half.

4. Sort the smaller arrays.
5. Merge the sorted arrays.
6. Continue until the complete array is sorted.
7. Display the sorted array.
8. Stop.

Python Program

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = 0
        j = 0
        k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
```

```
            arr[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Name: Farman Ilahi A")
print("Sorted Array:", arr)
```



Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω: O(nlogf_0n), O: O(n2), Θ: Θ(nlogf_0n)
- **Explanation:** Shows how pivot choice affects performance.

Answer:

Aim

To implement recursive quick sort and analyze its complexity.

Algorithm

1. Start.
2. Select a pivot element.
3. Divide the array into elements smaller and greater than the pivot.
4. Recursively sort the left part.

5. Recursively sort the right part.
6. Combine the sorted parts.
7. Display the sorted array.
8. Stop.

Python Program

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
 pivot = arr[-1]
    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)
arr = [10, 7, 8, 9, 1, 5]
print("Name: Farman Ilahi A")
print("Sorted Array:", quick_sort(arr))
```

Output

Sorted Array: [1, 5, 7, 8, 9, 10]

main.py Visualize Run ▶ Output

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)


arr = [10, 7, 8, 9, 1, 5]

print("Name: Farman Ilahi A")
print("Sorted Array:", quick_sort(arr))
```

```
Name: Farman Ilahi A
Sorted Array: [1, 5, 7, 8, 9, 10]
```

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** O(2n)
- **Explanation:** Illustrates exponential growth in recursive calls.

Answer:

Aim

To analyze exponential recursion using the Tower of Hanoi problem.

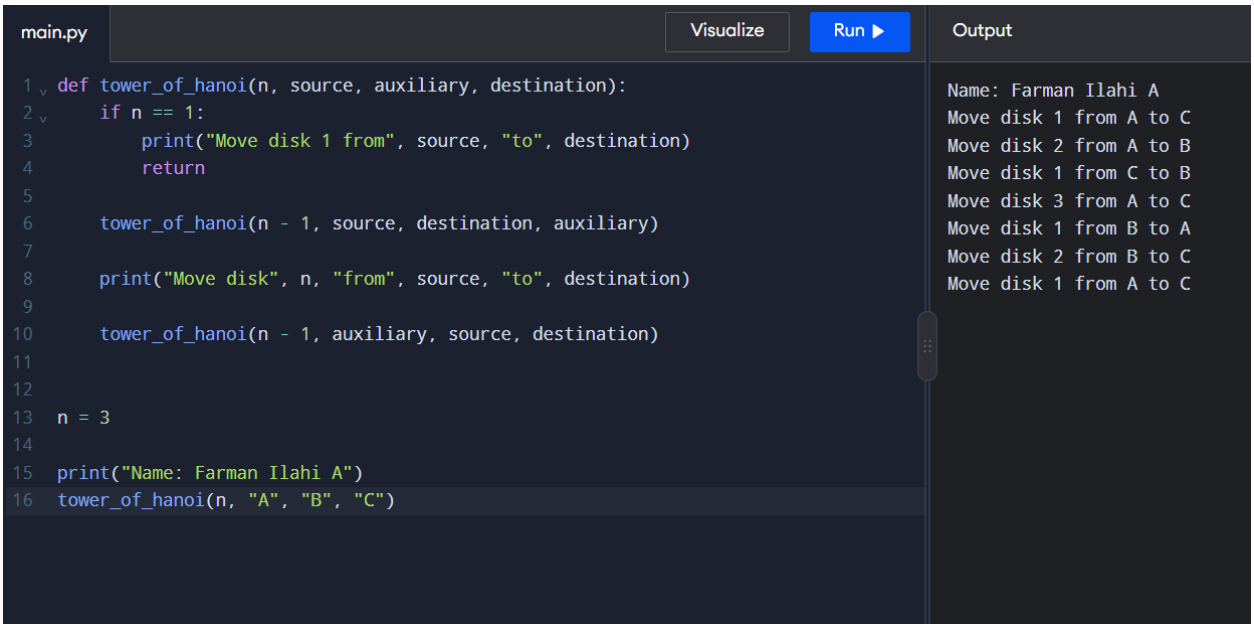
Algorithm

1. Start.
2. If there is only one disk, move it directly.

3. Move n-1 disks from source to auxiliary.
4. Move the largest disk from source to destination.
5. Move n-1 disks from auxiliary to destination.
6. Repeat recursively.
7. Stop.

Python Program

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return
    tower_of_hanoi(n - 1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    tower_of_hanoi(n - 1, auxiliary, source, destination)
n = 3
print("Name: Farman Ilahi A")
tower_of_hanoi(n, "A", "B", "C")
```



Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** O(log$_{fo}$n)
- **Explanation:** Shows efficient recursion with logarithmic complexity.

Answer:

Aim

To implement the recursive Euclidean algorithm for finding GCD.

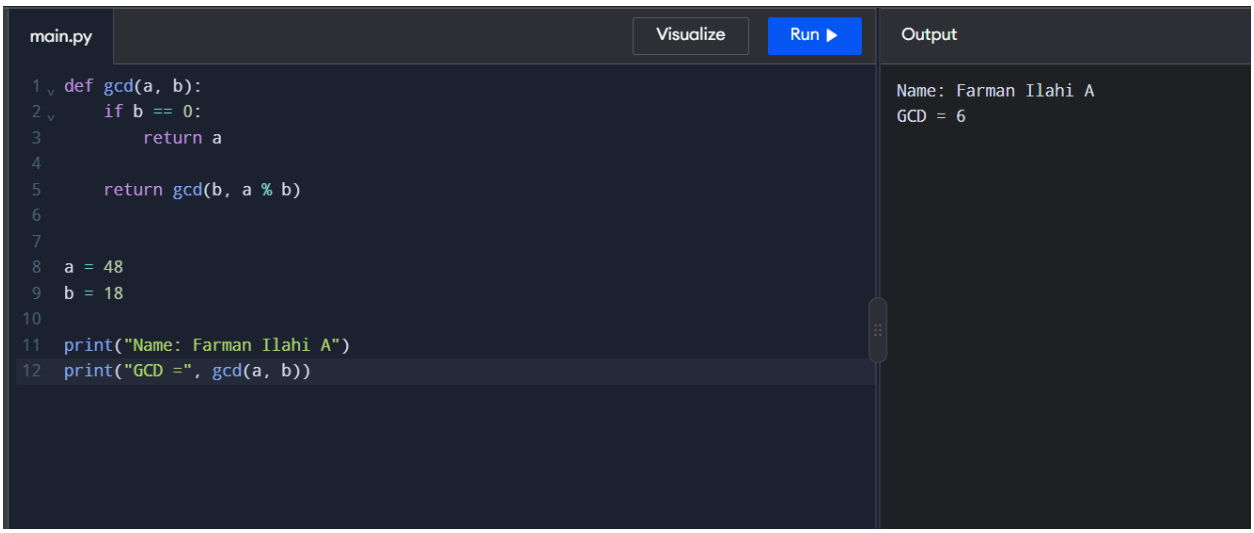
Algorithm

1. Start.
2. Read two numbers a and b.

3. If b = 0, return a.
4. Otherwise calculate a % b.
5. Recursively call GCD with b and a % b.
6. Display the GCD.
7. Stop.

Python Program

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
a = 48
b = 18
print("Name: Farman Ilahi A")
print("GCD =", gcd(a, b))
```



Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω: O(n), O: O(n2), Θ: Θ(n2)
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

Answer:

Aim

To implement insertion sort and analyze its time complexity.

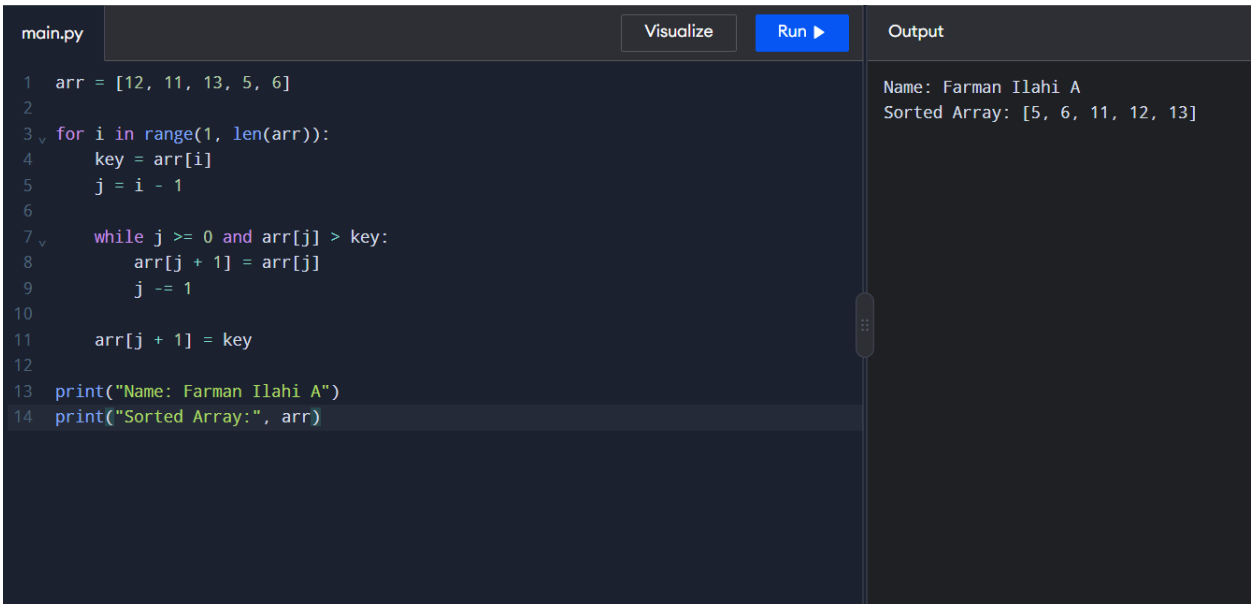
Algorithm

1. Start.
2. Assume the first element is sorted.
3. Select the next element as the key.
4. Compare it with the elements before it.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat until the array is sorted.
8. Display the sorted array.
9. Stop.

Python Program

```
arr = [12, 11, 13, 5, 6]
```

```
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = key
print("Name: Farman Ilahi A")
print("Sorted Array:", arr)
```



Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always O(nlog⁡n)
- **Explanation:** Consistent performance using heap structure.

Answer:

Aim

To implement heap sort and analyze heap-based sorting.

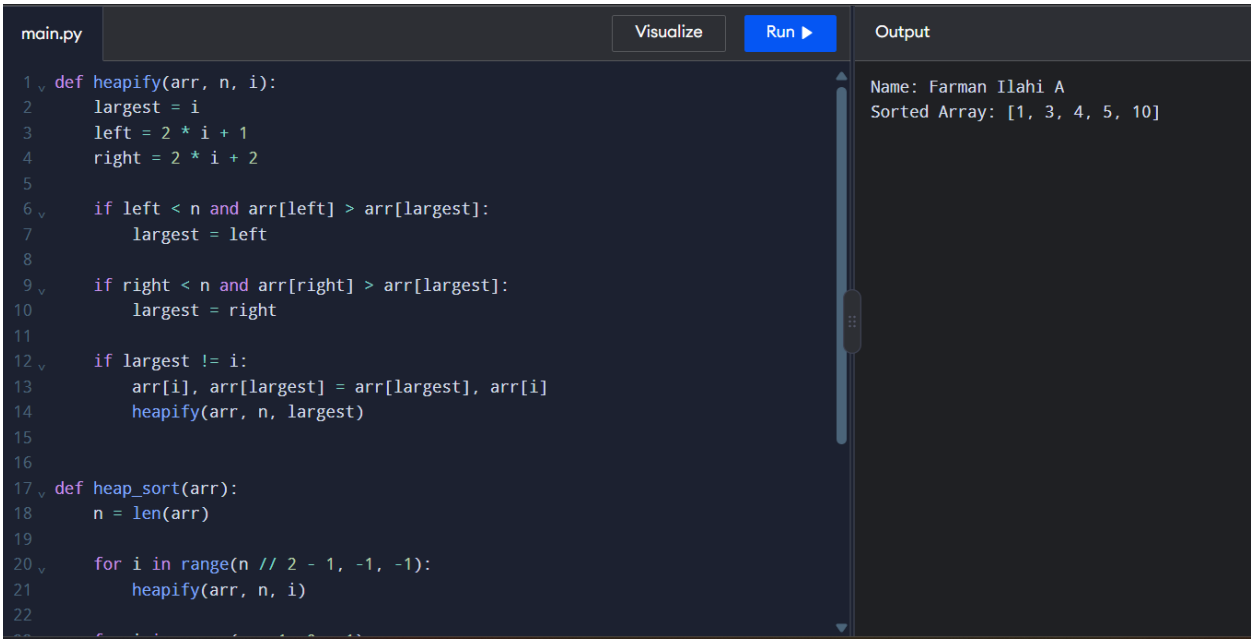
Algorithm

1. Start.
2. Build a max heap from the array.
3. Swap the root with the last element.
4. Reduce the heap size.
5. Heapify the root again.
6. Repeat until the array is sorted.
7. Display the sorted array.
8. Stop.

Python Program

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
```

```
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
arr = [4, 10, 3, 5, 1]
heap_sort(arr)
print("Name: Farman Ilahi A")
print("Sorted Array:", arr)
```



Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** O(nlogf_0n)
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

Answer:

Aim

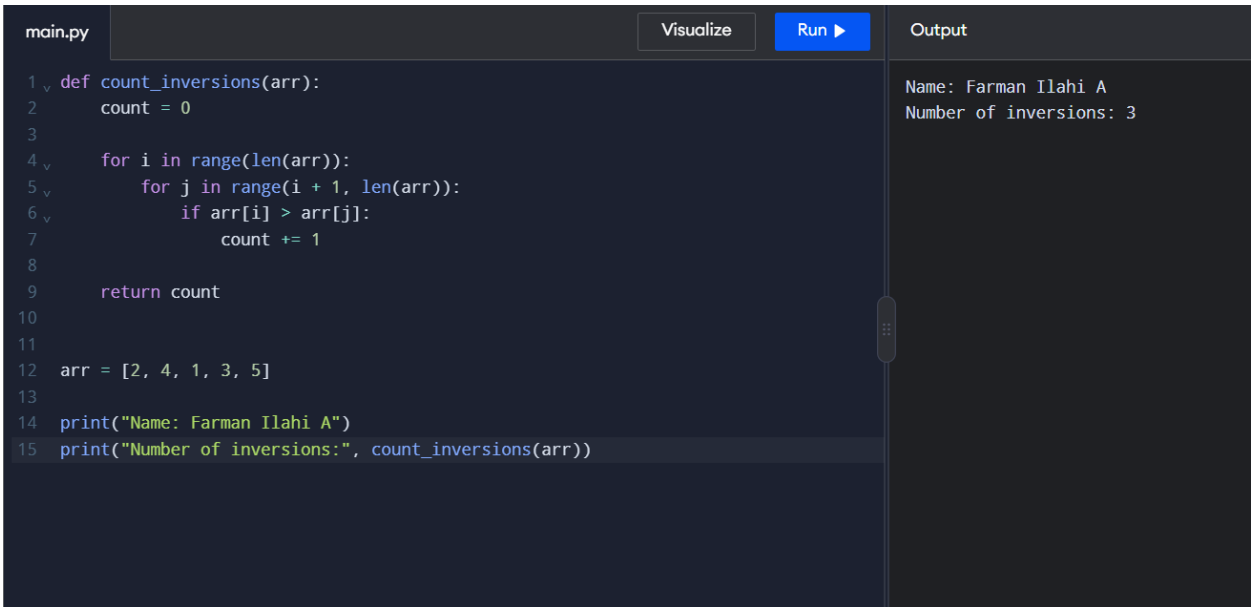
To count the number of inversions in an array.

Algorithm

1. Start.
2. Take each element of the array.
3. Compare it with all elements after it.
4. If the first element is greater than the second, increment the inversion count.
5. Continue until all pairs are checked.
6. Display the inversion count.
7. Stop.

Python Program

```
def count_inversions(arr):
    count = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                count += 1
    return count
arr = [2, 4, 1, 3, 5]
print("Name: Farman Ilahi A")
print("Number of inversions:", count_inversions(arr))
```



Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's O(n), Divide & Conquer O(nlogf_0n)
- **Explanation:** Highlights efficiency differences between approaches.

Answer:

Aim

To find the maximum subarray sum using Kadane's algorithm.

Algorithm

1. Start.
2. Initialize current_sum and max_sum.
3. Traverse the array.
4. Add the current element to current_sum.

5. Choose the maximum between the current element and the current sum.
6. Update max_sum.
7. Display the maximum subarray sum.
8. Stop.

Python Program

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]

current_sum = arr[0]

max_sum = arr[0]

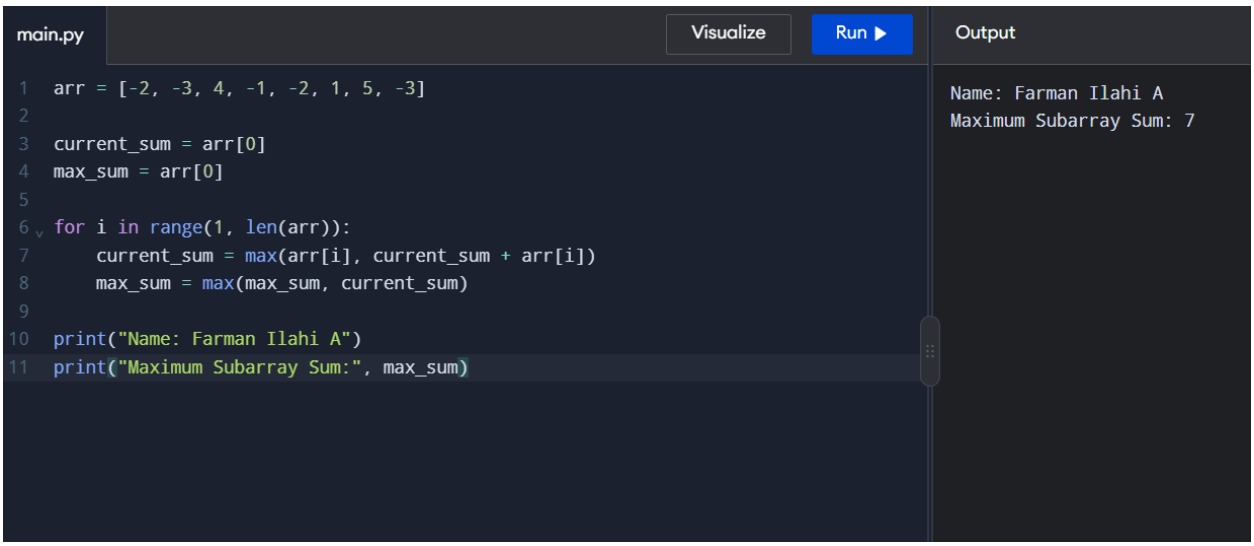
for i in range(1, len(arr)):

    current_sum = max(arr[i], current_sum + arr[i])

    max_sum = max(max_sum, current_sum)

print("Name: Farman Ilahi A")

print("Maximum Subarray Sum:", max_sum)
```



Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute xn.
- **Sample Input:** x=2, n=10
- **Sample Output:** 1024
- **Complexity:** Iterative O(n), Recursive fast power O(log n)
- **Explanation:** Shows how recursion reduces complexity.

Answer:

Aim

To compare iterative exponentiation with recursive fast power.

Algorithm

1. Start.
2. Read x and n.
3. If n = 0, return 1.
4. Calculate the power of n/2 recursively.
5. If n is even, square the result.
6. If n is odd, multiply by x.
7. Display the result.
8. Stop.

Python Program

```
def fast_power(x, n):
    if n == 0:
        return 1
```

```
    half = fast_power(x, n // 2)
    if n % 2 == 0:
        return half * half
    else:
        return x * half * half
x = 2
n = 10
print("Name: Farman Ilahi A")
print(x, "^", n, "=", fast_power(x, n))
```



Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)-(3,1), Distance = √5
- **Complexity:** Brute force O(n2), Optimized O(nlog n)
- **Explanation:** Demonstrates quadratic complexity and motivates optimization

Answer:

Aim

To find the closest pair of points using the brute-force approach.

Algorithm

1. **Start.**
2. **Store the given points.**
3. **Set the minimum distance to infinity.**
4. **Compare every pair of points.**
5. **Calculate the Euclidean distance between each pair.**
6. **If the distance is smaller than the current minimum, update the closest pair.**
7. **Display the closest pair and distance.**
8. **Stop.**

Python Program

```
import math
```

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]


min_distance = float("inf")
closest_pair = None


for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]
        distance = math.sqrt(
            (x1 - x2) ** 2 +
            (y1 - y2) ** 2
        )
        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])
print("Name: Farman Ilahi A")
print("Closest pair:", closest_pair)
print("Distance =", round(min_distance, 2))
```

main.py

Visualize

Run ▶

```python
points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_distance = float("inf")
closest_pair = None

for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        distance = math.sqrt(
            (x1 - x2) ** 2 +
            (y1 - y2) ** 2
        )

        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

print("Name: Farman Ilahi A")
print("Closest pair:", closest_pair)
print("Distance =", round(min_distance, 2))
```

Output

```
Name: Farman Ilahi A
Closest pair: ((1, 2), (3, 1))
Distance = 2.24
```